INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
DE SÃO PAULO

CÂMPUS GUARULHOS

GUSTAVO DA SILVA COELHO

RAISSA CARLA FERREIRA DA SILVA

Sistema de apoio à análise de dados com processamento de linguagem natural (NLP)

GUARULHOS

ANO

GUSTAVO DA SILVA COELHO
RAISSA CARLA FERREIRA DA SILVA

Sistema de apoio à análise de dados com processamento de linguagem natural (NLP)

Relatório Técnico apresentado ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, como parte dos requisitos para a obtenção do grau de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Orientador: Prof. Me. Giovani Fonseca Ravagnani Disperati

GUARULHOS

ANO

Catalogação na fonte
Biblioteca IFSP Câmpus Guarulhos
Dados fornecidos pelo(a) autor(a)

_ERROCOD ,
IGOCUTTE R_ _ / _. Guarulhos: [s.n.], 2024.
100 f. il.

Orientador: _
Co-orientador: _

Trabalho de Conclusão de Curso (Tecnologia em Análise e Desenvolvimento de Sistemas) - Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, IFSP, 2024.

Inclui bibliografia.

1. _. 2. _. 3. _. 4. _. 5. _. I. Instituto Federal de Educação, Ciência e Tecnologia de São Paulo II. Título.

CDD 004.21

ATA N.º /2024 - DAE-GRU/DRG/GRU/IFSP

ATA DE DEFESA DE MONOGRAFIA

Na presente data realizou-se a sessão pública de defesa da Monografia intitulada

Membros	IES	Presença (Sim/Não)	Aprovação/Conceito *(Quando Exigido)*

Observações:

A banca examinadora, tendo terminado a apresentação do conteúdo da monografia, passou à arguição do candidato. Em seguida, os examinadores reuniram-se para avaliação e deram o parecer final sobre o trabalho apresentado pelo aluno, tendo sido atribuído o seguinte resultado:

[ ] Aprovado [ ] Reprovado

Proclamados os resultados pelo presidente da banca examinadora, foram encerrados os trabalhos e, para constar, eu lavrei a presente ata que assino juntamente com os demais membros da banca examinadora.

Campus Guarulhos

Documento assinado eletronicamente.

Documento assinado eletronicamente por:

-
-
-
-

Este documento foi emitido pelo SUAP em 05/07/2024. Para comprovar sua autenticidade, faça a leitura do QRCode ao lado ou acesse https://suap.ifsp.edu.br/autenticar-documento/ e forneça os dados abaixo:

Código Verificador:
Código de Autenticação:

Dedicatória (Opcional). Não digite a palavra Dedicatória. Texto no qual o autor do trabalho oferece homenagem ou dedica o seu trabalho a alguém (não usar ponto final)

AGRADECIMENTOS

Agradecimento (opcional). Folha que contém manifestação de reconhecimento a pessoas e/ou instituições que realmente contribuíram com o autor.

Epígrafe (Opcional). Pensamentos retirados de um livro, uma música, um poema, normalmente relacionado ao tema do trabalho, seguida de indicação de autoria. As epígrafes podem ser colocadas também nas folhas de abertura de cada capítulo.

"*Here's to the crazy ones. The misfits. The rebels. The troublemakers. The round pegs in the square holes. The ones who see things differently. They're not fond of rules. And they have no respect for the status quo. You can quote them, disagree with them, glorify or vilify them. About the only thing you can't do is ignore them. Because they change things. They push the human race forward. And while some may see them as the crazy ones, we see genius. Because the people who are crazy enough to think they can change the world, are the ones who do.*".

RESUMO

Elemento obrigatório, constituído de uma sequência de frases concisas e objetivas, fornecendo uma visão rápida e clara do conteúdo do estudo. O texto deverá conter entre 150 a 250 palavras e ser antecedido pela referência do estudo. Também, não deve conter citações e deverá ressaltar o objetivo, o método, os resultados e as conclusões. O resumo deve ser redigido em parágrafo único, seguido das palavras representativas do conteúdo do estudo, isto é, palavras-chave, em número de três a cinco, separadas entre si por ponto e finalizadas também por ponto. Usar o verbo na terceira pessoa do singular, com linguagem impessoal (pronome SE), bem como fazer uso, preferencialmente, da voz ativa.

Palavras-chave: palavra-chave 1; palavra-chave 2; palavra-chave 3; palavra-chave n.

LISTA DE ILUSTRAÇÕES

Figura 1 – Cronograma previsto de outubro de 2026 a junho de 2027 54
Figura 2 – Diagrama de casos de uso – fluxo central do MVP (V1) 65
Figura 3 – Diagrama de classes – núcleo de geração e validação previsto para a V1 74
Figura 4 – Diagrama de classes – contexto por conexão e execução prevista para a V1 76
Figura 5 – DER proposto para a persistência local SQLite da V1 84

LISTA DE QUADROS

Quadro 1 – Atores do sistema 63
Quadro 2 – Casos de uso e requisitos atendidos 65
Quadro 3 – Rastreabilidade dos requisitos funcionais 70
Quadro 4 – Critérios de verificação dos requisitos não funcionais 71

LISTA DE TABELAS

LISTA DE ABREVIATURAS E SIGLAS

1D	Uma dimensão
2D	Duas dimensões
3D	Três dimensões

LISTA DE SÍMBOLOS

α	Letra grega minúscula Alfa
β	Letra grega minúscula Beta

SUMÁRIO

1	INTRODUÇÃO	29
1.1	Justificativa	30
1.2	Objetivos	31
1.2.1	Objetivo Geral	31
1.2.2	Objetivos Específicos	31
2	MÉTODO DE DESENVOLVIMENTO	33
2.1	Visão Geral do Projeto	33
2.2	Requisitos	36
2.2.1	Requisitos Funcionais	36
2.2.2	Requisitos Não Funcionais	38
2.2.3	Não Escopo	39
2.3	Processo de Desenvolvimento de Software	40
2.4	Tecnologias	42
2.4.1	Python	42
2.4.2	FastAPI	43
2.4.3	Pydantic	43
2.4.4	PostgreSQL	43
2.4.5	Acesso ao PostgreSQL e Validação SQL	43
2.4.6	React, Vite e TypeScript	44
2.4.7	Electron	44
2.4.8	Modelos de Linguagem	45
2.4.9	Git e GitHub	45
2.4.10	Execução Local	46
3	RESULTADOS E DISCUSSÃO	47
4	CONCLUSÕES PARCIAIS	49
4.1	Papel do Contexto na Solução	49
4.2	Definições Consolidadas e Pendências	50
4.3	Maturidade do Escopo e Lições Aprendidas	50
4.4	Avaliação Prevista e Evoluções	51
5	CRONOGRAMA E GESTÃO DE PESSOAS	53
5.1	Gestão de Pessoas	53
5.2	Cronograma das Atividades	54
5.3	Planejamento da Avaliação	56

REFERÊNCIAS 59

APÊNDICES 61

APÊNDICE A – MODELO DE REQUISITOS DO SISTEMA 63
A.1 Atores 63
A.2 Regras de Negócio 63
A.3 Casos de Uso 64
A.3.1 UC01 – Configurar conexão com o banco 65
A.3.2 UC02 – Identificar estrutura do banco 66
A.3.3 UC03 – Gerenciar consentimento de amostragem 66
A.3.4 UC04 – Realizar consulta em linguagem natural 67
A.3.5 UC05 – Validar solicitação e consulta 68
A.3.6 UC06 – Executar consulta e apresentar resultado 68
A.3.7 UC07 – Corrigir consulta automaticamente 68
A.3.8 UC08 – Tratar falha 69
A.3.9 UC09 – Preparar contexto de negócio 69
A.3.10 UC10 – Configurar perfil de IA 69
A.3.11 UC11 – Consultar histórico e preferências 69
A.3.12 UC12 – Acessar conta do produto 69
A.4 Rastreabilidade dos Requisitos 70
A.5 Critérios de Verificação dos Requisitos Não Funcionais 70

APÊNDICE B – MODELO ESTRUTURAL DO SISTEMA 73
B.1 Núcleo de Geração e Validação SQL 73
B.2 Contexto da Conexão e Execução Confirmada 75
B.3 Considerações sobre o Modelo 77

APÊNDICE C – MODELO COMPORTAMENTAL DO SISTEMA . . 79

APÊNDICE D – MODELO DA ARQUITETURA DO SISTEMA . . . 81

APÊNDICE E – MODELO DE DADOS DO SISTEMA 83

APÊNDICE F – OUTRO APÊNDICE 87

ANEXOS 89

ANEXO A – TÍTULO DO ANEXO A 91

ANEXO B – TÍTULO DO ANEXO B 93

ANEXO C – TÍTULO DO ANEXO C 95

ANEXO D – TÍTULO DO ANEXO D 97

ANEXO E – TÍTULO DO ANEXO E 99

1 INTRODUÇÃO

O acesso rápido a informações confiáveis é importante para o acompanhamento de processos e para a tomada de decisões nas organizações. Entretanto, os dados corporativos costumam estar armazenados em bancos de dados relacionais cuja consulta exige conhecimento de linguagens específicas, como a Structured Query Language (SQL). Em consequência, colaboradores que conhecem o contexto do negócio, mas não dominam SQL, frequentemente dependem de profissionais de Tecnologia da Informação (TI) para obter relatórios e responder a perguntas operacionais. Essa dependência pode aumentar o tempo necessário para acessar informações e direcionar a equipe técnica a solicitações repetitivas de extração de dados.

Os avanços em Inteligência Artificial (IA), especialmente no Processamento de Linguagem Natural (PLN), ampliaram as possibilidades de interação entre pessoas e sistemas computacionais. Nesse contexto, a área denominada *text-to-SQL* investiga métodos capazes de interpretar uma pergunta formulada em linguagem natural e transformá-la em uma consulta SQL compatível com a estrutura de um banco de dados (Deng; Chen; Zhang, 2022). Essa abordagem pode tornar a consulta de dados mais acessível, mas ainda envolve desafios relacionados à compreensão da intenção do usuário, à associação entre os termos da pergunta e o esquema do banco, à correção da consulta produzida e à generalização para diferentes estruturas de dados (Yu et al., 2018).

Este trabalho tem como tema a aplicação de IA na consulta a bancos de dados e delimita-se ao desenvolvimento de uma aplicação desktop para consultas a bancos de dados PostgreSQL. Por meio da plataforma, o usuário poderá formular, em português e sem escrever SQL, perguntas como “Quais foram os cinco produtos mais vendidos em julho?”. A aplicação enviará a pergunta e as informações necessárias sobre o esquema do banco a um modelo de IA, receberá uma consulta SQL, verificará sua sintaxe e sua segurança, executará apenas consultas de leitura validadas e explicitamente confirmadas pelo usuário e apresentará o resultado de forma compreensível.

O problema que orienta o projeto pode ser expresso pela seguinte questão: como permitir que usuários sem conhecimento técnico em SQL consultem dados relacionais por meio de linguagem natural, preservando a segurança, a integridade e o controle de acesso às informações? Para responder a essa questão, a solução proposta combinará geração de SQL por IA com uma camada de validação independente da resposta do modelo. Serão aceitas exclusivamente consultas do tipo `SELECT`, e comandos capazes de alterar estruturas ou registros, como `INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER` e `TRUNCATE`, deverão ser bloqueados antes da execução.

O MVP contemplará cadastro e login da conta do produto, configuração de conexões PostgreSQL, escolha entre Ollama local e API externa com chave do próprio usuário, histórico local de conversas e apresentação dos resultados em tabelas. A preparação do contexto combinará a estrutura acessível do banco com regras de negócio confirmadas pelo usuário em uma entrevista inicial por conexão. A interface utilizará React, Vite e TypeScript, distribuídos com Electron, e o *back-end* local será implementado em Python com FastAPI. Gráficos, explicações textuais dos resultados, favoritos, sugestões de perguntas e exportações avançadas não integrarão o compromisso da V1. O sistema será avaliado em cenários controlados e com bancos de dados de teste, sem permitir alterações nos dados consultados.

1.1 Justificativa

Sob a perspectiva teórica, o projeto se justifica pela relevância do problema *text-to-SQL* para as áreas de PLN e bancos de dados. A tradução de uma pergunta para SQL não corresponde apenas à substituição de palavras: o sistema precisa identificar a intenção expressa, relacionar entidades e atributos ao esquema disponível e construir uma consulta sintática e semanticamente adequada. A literatura apresenta avanços importantes nessa área, mas também reconhece desafios de interpretação, geração e avaliação das consultas (Deng; Chen; Zhang, 2022). O conjunto de dados Spider, por exemplo, foi elaborado para avaliar consultas complexas e a generalização entre diferentes domínios e esquemas de bancos de dados, evidenciando a necessidade de soluções que sejam testadas além de exemplos simples ou de uma única base (Yu et al., 2018).

Em termos acadêmicos, o desenvolvimento da plataforma possibilita integrar conhecimentos de engenharia de software, bancos de dados, segurança da informação, desenvolvimento web e IA em um problema aplicado. Além de construir um protótipo funcional, o trabalho permitirá observar em quais tipos de pergunta a geração automática produz resultados corretos, quais erros ocorrem com maior frequência e em que medida as verificações implementadas impedem consultas inadequadas. Assim, a avaliação do sistema poderá produzir evidências sobre as possibilidades e as limitações do uso de modelos de IA como interface para dados relacionais.

Sob a perspectiva prática e de mercado, a solução busca aproximar os dados dos profissionais que precisam utilizá-los, mas que não possuem formação técnica em SQL. Ao permitir consultas em linguagem cotidiana, a plataforma poderá reduzir o tempo gasto em solicitações simples de relatórios, dar maior autonomia aos usuários e liberar a equipe de TI para atividades de maior complexidade. A apresentação tabular, acompanhada da possibilidade de inspecionar a SQL gerada, poderá facilitar a conferência dos resultados e ampliar o uso das informações disponíveis no processo

decisório.

Por fim, a adoção de IA nesse contexto exige cuidados específicos, pois uma consulta incorreta pode apresentar resultados enganosos, consumir recursos excessivos ou afetar informações sensíveis. Por esse motivo, a proposta não tratará a resposta do modelo como uma instrução automaticamente confiável. A restrição a operações de leitura, a validação prévia do SQL, o controle de permissões, o registro do histórico e o uso de bancos de teste constituem partes centrais da solução. Dessa maneira, o projeto procura conciliar facilidade de uso e inovação com mecanismos objetivos de segurança e rastreabilidade.

1.2 Objetivos

1.2.1 Objetivo Geral

Desenvolver e avaliar um protótipo de aplicação desktop que permita a usuários sem domínio de SQL consultar bancos de dados PostgreSQL por meio de perguntas em linguagem natural, utilizando IA para gerar consultas de leitura, validando-as e solicitando confirmação explícita antes da execução e apresentando os resultados de maneira segura e compreensível.

1.2.2 Objetivos Específicos

- Revisar conceitos, técnicas e trabalhos relacionados a PLN, modelos de IA e conversão de linguagem natural para SQL;
- Definir a arquitetura da plataforma e o fluxo de comunicação entre interface desktop, *back-end*, modelo de IA e banco de dados;
- Desenvolver uma interface desktop intuitiva com tecnologias web para o envio de perguntas e a visualização das respostas, considerando usuários sem conhecimento técnico em SQL;
- Implementar o *back-end* majoritariamente em Python, adotando FastAPI como camada de API local;
- Integrar modelos de IA por Ollama local e por API externa do usuário, capazes de gerar SQL a partir de perguntas em português, do esquema autorizado e do contexto de negócio da conexão;
- Implementar uma camada independente de validação sintática e de segurança que autorize somente consultas de leitura e bloqueie comandos de alteração de dados ou de estrutura;

- Implementar cadastro e login da conta do produto, respeitar as permissões da role PostgreSQL preexistente e persistir configurações e histórico de conversas localmente;
- Apresentar a SQL para inspeção e confirmação explícita do usuário e exibir os resultados em tabelas;
- Conduzir uma entrevista inicial para registrar regras de negócio validadas em um arquivo de contexto por conexão e permitir a correção controlada de SQL dentro do limite de três tentativas totais; e
- Avaliar o protótipo em bancos de dados de teste, considerando a correção das consultas geradas, a efetividade das regras de segurança, o tempo de execução e a compreensão dos resultados apresentados.

2 MÉTODO DE DESENVOLVIMENTO

Este capítulo apresenta o método de desenvolvimento adotado para a construção do sistema proposto, contemplando a visão geral da solução, seus requisitos funcionais e não funcionais, o processo de desenvolvimento de software e as principais tecnologias previstas.

O projeto será conduzido de maneira incremental, priorizando inicialmente a validação do fluxo central da solução: identificação automática da estrutura de um banco de dados PostgreSQL, interpretação de uma pergunta formulada em linguagem natural, geração de uma consulta SQL segura, execução da consulta e apresentação do resultado ao usuário. A partir dessa base, funcionalidades adicionais poderão ser incorporadas progressivamente.

A solução será desenvolvida por uma equipe composta por dois integrantes, utilizando um processo baseado em Kanban e adaptado às características e à escala do projeto acadêmico. O desenvolvimento será acompanhado por meio de um backlog, histórias de usuário, reuniões periódicas e validações progressivas dos incrementos produzidos.

2.1 Visão Geral do Projeto

O projeto propõe o desenvolvimento de uma aplicação capaz de permitir consultas a bancos de dados PostgreSQL por meio de linguagem natural. O usuário deverá fornecer as informações necessárias para conexão com um banco de dados e, posteriormente, poderá formular perguntas textuais relacionadas aos dados armazenados. A aplicação será responsável por interpretar a solicitação, gerar uma consulta SQL compatível com a estrutura identificada, executá-la de maneira controlada e apresentar o resultado obtido.

Um dos principais componentes da solução será o módulo de descoberta e introspecção do banco de dados, denominado internamente como *crawler*. Esse módulo realizará automaticamente o levantamento da estrutura acessível por meio da conexão fornecida pelo usuário, percorrendo os esquemas disponíveis e identificando, no mínimo, tabelas, colunas, tipos de dados, chaves primárias e chaves estrangeiras.

A descoberta estrutural será realizada de maneira determinística, sem depender diretamente de um modelo de linguagem para identificar os elementos básicos do banco de dados. As informações encontradas serão organizadas em uma representação estruturada e posteriormente utilizadas como contexto pelas etapas responsáveis pela

interpretação da solicitação e pela geração da consulta SQL.

Além dos metadados estruturais, o sistema poderá utilizar pequenas amostras dos registros armazenados para auxiliar na compreensão do significado prático das tabelas e colunas. Como essas amostras podem conter informações sensíveis, sua utilização dependerá de consentimento explícito do usuário.

Quando autorizada, a amostragem será limitada a cem registros por tabela, abrangendo todas as colunas acessíveis à role. A autorização será concedida por tabela, com decisão separada para eventual envio a um provedor remoto. Caso o usuário não autorize amostras, o sistema continuará funcionando com os metadados estruturais e o contexto de negócio permitido.

De maneira geral, a arquitetura conceitual da solução será composta pelo seguinte fluxo:

Interface desktop $\rightarrow$ Preparação do contexto da conexão $\rightarrow$ Geração de SQL $\rightarrow$ Validação independente $\rightarrow$ Inspeção e confirmação pelo usuário $\rightarrow$ Execução somente leitura $\rightarrow$ Resultado tabular.

A aplicação será distribuída como software desktop para Windows, com interface Electron/React e backend Python/FastAPI incluído no instalador e iniciado automaticamente. O núcleo de consulta será executado no computador do usuário, permitindo acesso a PostgreSQL local ou de rede interna. Um serviço separado em nuvem, denominado *Control Plane*, atenderá cadastro, login, registro de contas e instalações e métricas básicas agregadas, sem receber perguntas, SQLs, esquemas, regras de negócio ou resultados. A política de sessão e operação offline desse serviço permanece pendente; sua indisponibilidade não deverá exigir dependência contínua da nuvem para o uso local de Ollama ou da API externa do usuário.

A etapa de interpretação da solicitação também será utilizada para identificar pedidos incompatíveis com o propósito da aplicação. Solicitações que envolvam inserção, atualização, exclusão ou alteração estrutural do banco não deverão prosseguir para execução.

O desenvolvimento será organizado em três marcos de entrega: demonstração de 20%, prova de conceito V0 e MVP desktop V1. O marco de 20%, entregue na semana de 2 a 8 de novembro de 2026, demonstrou conexão com PostgreSQL em Docker, introspecção autorizada, geração via Ollama e validação inicial, sem execução da SQL proposta. A V0 está prevista para a semana de 9 a 15 de novembro de 2026, e a V1 para o semestre seguinte.

A **V0** corresponderá à prova de conceito por interface de linha de comando (CLI), reutilizando o núcleo dos 20%. Acrescentará contexto de negócio por conexão, até

três tentativas totais de geração, confirmação explícita da SQL e execução controlada de leitura com apresentação tabular no terminal. A entrevista poderá ser explorada experimentalmente, sem exigir o fluxo completo da V1.

A **V1** corresponderá ao Produto Mínimo Viável (MVP) e constituirá a entrega-alvo do trabalho. Integrará o núcleo a uma interface desktop, com múltiplas conexões configuráveis, Ollama local e ao menos uma API externa do usuário, entrevista de contexto por conexão, histórico e configurações locais, proteção opcional de segredos e Control Plane básico. Os resultados serão apresentados em formato tabular.

A **V2** será considerada uma evolução prevista da solução e acrescentará uma etapa de interpretação dos resultados obtidos, permitindo que o sistema apresente também uma explicação textual dos dados retornados.

Por fim, a **V3** representará uma evolução voltada à visualização de dados. Nessa etapa, o sistema poderá sugerir formas adequadas de representação gráfica dos resultados, enquanto a interface será responsável pela renderização dessas visualizações.

As versões V2 e V3 serão tratadas como possibilidades de evolução e não como requisitos obrigatórios para a conclusão do MVP.

A unidade de configuração e isolamento será a conexão, incluindo o destino PostgreSQL e a identidade de acesso. Cada conexão terá um único arquivo `context.md`, com regras de negócio confirmadas pelo usuário. Na V1, a entrevista inicial poderá formular até quinze perguntas, com alternativas sugeridas e resposta livre; hipóteses não confirmadas não serão incorporadas como regras. Na V0, o arquivo poderá ser preparado manualmente.

O `ContextBuilder` combinará políticas técnicas, esquema autorizado, arquivo de contexto completo, pergunta atual e pares anteriores de pergunta/SQL da mesma sessão. Não haverá RAG, embeddings ou seleção semântica de partes da memória. O orçamento inicial será de oito mil tokens para a memória de negócio e dezesseis mil para o contexto de entrada completo, respeitando a janela do modelo e a reserva de saída; excessos serão informados, sem descarte silencioso de regras. Linhas retornadas por consultas não alimentarão a LLM.

Na V1, o SQLite persistirá apenas dados operacionais locais: perfil e preferências do usuário, configurações de conexões e de IA, consentimentos necessários e histórico de conversas. O conhecimento de negócio permanecerá em Markdown, e senhas e chaves salvas opcionalmente serão protegidas por Electron safeStorage/DPAPI no Windows, com referências no SQLite. Não serão replicados resultados, amostras ou o catálogo completo do PostgreSQL. Na V0, configurações e contexto poderão usar arquivos simples, sem persistência de segredos.

2.2 Requisitos

Os requisitos de software representam os serviços que o sistema deverá oferecer, bem como as restrições que deverão ser consideradas em seu desenvolvimento. Segundo (Sommerville, 2003; Sommerville, 2007), os requisitos podem ser classificados em funcionais, relacionados diretamente às funcionalidades e comportamentos esperados do sistema, e não funcionais, associados a restrições e propriedades de qualidade da solução.

Além desses requisitos, também será explicitado o não escopo do projeto, delimitando funcionalidades que não serão consideradas parte da entrega prevista. A definição dessas fronteiras contribui para o controle da complexidade e para a manutenção dos objetivos estabelecidos para o projeto (PRESSMAN, 2016).

2.2.1 Requisitos Funcionais

Os requisitos funcionais inicialmente definidos para a aplicação são apresentados a seguir:

- **RF01 – Conexão com banco de dados:** o sistema deverá permitir configurar e testar conexões PostgreSQL com roles preexistentes e permissões restritas, sem provisionamento administrativo; a V1 permitirá manter múltiplas configurações, com uma conexão ativa por conversa.
- **RF02 – Identificação de esquemas:** o sistema deverá identificar automaticamente os esquemas acessíveis utilizando a conexão fornecida.
- **RF03 – Introspecção estrutural:** o sistema deverá identificar automaticamente tabelas, colunas, tipos de dados, chaves primárias e chaves estrangeiras existentes nas estruturas acessíveis.
- **RF04 – Consentimento para amostragem:** o sistema deverá solicitar autorização explícita por tabela para usar amostras, com consentimento separado para envio a um provedor remoto.
- **RF05 – Limitação das amostras:** quando a utilização de registros for autorizada, o sistema deverá limitar a amostragem a cem registros por tabela, incluindo todas as colunas acessíveis à role.
- **RF06 – Entrada em linguagem natural:** o sistema deverá permitir que o usuário formule perguntas sobre os dados utilizando linguagem natural.
- **RF07 – Validação da intenção:** o sistema deverá identificar solicitações incompatíveis com o escopo exclusivamente de leitura da aplicação.

- **RF08 – Geração de consulta SQL:** o sistema deverá gerar consultas SQL de leitura a partir da pergunta, do esquema autorizado e do contexto de negócio validado da conexão ativa.

- **RF09 – Execução da consulta:** o sistema deverá apresentar a SQL validada e executá-la somente após confirmação explícita vinculada à SQL exata e à conexão. A proposta será revalidada antes da execução, em transação somente leitura, com timeout de trinta segundos e limite de linhas; alterações na SQL exigirão nova confirmação.

- **RF10 – Apresentação do resultado:** o sistema deverá apresentar o resultado obtido em formato tabular por meio da interface gráfica da aplicação.

- **RF11 – Correção automática de consultas:** quando uma consulta considerada permitida apresentar falha durante sua geração ou execução, o sistema poderá realizar até três tentativas totais de geração: uma inicial e até duas correções. Cada nova SQL será validada e, para execução, exigirá nova confirmação explícita.

- **RF12 – Tratamento de falha:** caso a consulta não possa ser concluída após o limite de tentativas estabelecido, o sistema deverá encerrar a operação de maneira controlada e informar a falha ao usuário.

- **RF13 – Contexto de negócio por conexão:** o sistema deverá carregar integralmente um `context.md` por conexão; na V1, uma entrevista de até quinze perguntas permitirá confirmar, corrigir ou rejeitar hipóteses antes de gravar as regras, que poderão ser revistas pelo usuário.

- **RF14 – Configuração de IA:** a V1 deverá permitir configurar e testar perfis de Ollama local e de ao menos uma API externa com chave do próprio usuário.

- **RF15 – Persistência local:** a V1 deverá manter histórico de conversas com perguntas, SQLs e estados de validação, além de preferências, configurações e consentimentos essenciais. Resultados tabulares não integrarão o histórico persistente por padrão.

- **RF16 – Conta e instalação:** a V1 deverá oferecer cadastro e login e registrar conta e instalação no Control Plane, com métricas agregadas que não incluam o conteúdo das consultas ou do contexto.

Como requisitos evolutivos, posteriores ao MVP, estarão previstos a geração de explicações textuais sobre os resultados na V2 e a geração ou sugestão de visualizações gráficas na V3.

2.2.2 Requisitos Não Funcionais

Os principais requisitos não funcionais definidos são:

- **RNF01 – Segurança:** o sistema deverá operar exclusivamente com consultas de leitura, bloqueando escrita e alteração estrutural por validação sintática com SQLGlot e política conservadora, combinadas com role de menor privilégio, transação somente leitura, timeout e limites de retorno. A LLM não receberá credenciais nem autoridade de execução.
- **RNF02 – Privacidade:** amostras somente poderão compor o contexto mediante autorização por tabela e consentimento separado para envio remoto. Resultados de consultas não serão enviados à LLM; segredos não serão incluídos em prompts ou logs.
- **RNF03 – Independência da amostragem:** a recusa do usuário em disponibilizar amostras de registros não poderá impedir o funcionamento básico da aplicação, que deverá permanecer capaz de operar utilizando apenas os metadados estruturais do banco.
- **RNF04 – Modularidade:** a arquitetura deverá manter separadas as responsabilidades relacionadas à introspecção do banco, preparação de contexto, geração de SQL, validação, execução e apresentação dos resultados.
- **RNF05 – Baixo acoplamento ao modelo de linguagem:** a integração com inteligência artificial deverá ser projetada de maneira que permita a substituição do modelo ou do provedor sem exigir a reestruturação completa da aplicação.
- **RNF06 – Tratamento de erros:** falhas relacionadas à conexão com o banco, comunicação com o provedor de inteligência artificial, geração da consulta ou execução do SQL deverão ser tratadas de forma controlada.
- **RNF07 – Observabilidade:** o sistema deverá registrar informações suficientes para permitir a análise de seu comportamento durante os testes, incluindo latência, categorias de erro, quantidade de tentativas e estado da execução, sem registrar indiscriminadamente dados pessoais, segredos ou linhas de resultado.
- **RNF08 – Usabilidade:** a V1 deverá disponibilizar uma interface gráfica executável no computador do usuário, permitindo realizar o fluxo principal da aplicação sem necessidade de interação direta com código-fonte ou terminal.

Durante a validação da aplicação também serão consideradas métricas relacionadas ao comportamento operacional do sistema. Entre elas estarão a latência entre a

solicitação e a resposta, a ocorrência de erros, o volume de consultas processadas e possíveis limitações de recursos.

Devido ao caráter local e à baixa concorrência esperada para o MVP, as métricas de latência e erros possuirão maior relevância inicial. Também serão avaliadas métricas específicas relacionadas à qualidade da solução, tais como taxa de respostas corretas, taxa de execução SQL bem-sucedida, taxa de bloqueio correto de operações proibidas, quantidade de consultas que necessitaram de novas tentativas e número médio de tentativas realizadas.

2.2.3 Não Escopo

Com o objetivo de manter o desenvolvimento compatível com o tempo, tamanho da equipe e objetivos acadêmicos do projeto, algumas funcionalidades serão explicitamente excluídas do escopo obrigatório.

Não fazem parte do escopo do MVP:

- inserção de novos registros no banco de dados;
- atualização de registros existentes;
- exclusão de registros;
- criação, alteração ou remoção de tabelas, esquemas ou demais objetos estruturais;
- suporte obrigatório a sistemas gerenciadores de bancos de dados diferentes do PostgreSQL;
- suporte inicial a bancos de dados NoSQL;
- criação de dashboards completos;
- treinamento de um modelo de linguagem próprio;
- hospedagem do núcleo de consulta em nuvem ou disponibilização da interface como aplicação web pública; o Control Plane básico será um serviço separado da V1;
- construção de catálogo semântico relacional ou banco vetorial; o arquivo de contexto validado por conexão fará parte do escopo;
- compartilhamento automático de regras entre conexões, consultas federadas e atualização automática do catálogo;

- RAG, embeddings, gráficos, explicações textuais de resultados, exportações avançadas, cobrança e API intermediária paga do AskDB.

Catálogos semânticos estruturados e mecanismos de recuperação poderão ser avaliados em evoluções posteriores. Essa exclusão não abrange os termos e regras confirmados no arquivo de contexto da conexão, que integram a V0 e a V1.

2.3 Processo de Desenvolvimento de Software

O desenvolvimento da aplicação será realizado utilizando uma abordagem incremental e iterativa, apoiada por um processo baseado em Kanban e adaptado ao tamanho reduzido da equipe e às características acadêmicas do projeto.

A equipe será composta por dois integrantes. Dessa forma, optou-se por um processo com baixa sobrecarga de gestão e documentação, privilegiando visibilidade das atividades, priorização contínua e execução progressiva das funcionalidades.

O trabalho será organizado a partir de um backlog contendo histórias de usuário, atividades técnicas, correções e demais tarefas necessárias para o desenvolvimento. Os itens serão priorizados conforme sua relevância para o funcionamento da solução e serão selecionados de acordo com a capacidade disponível da equipe.

Serão realizadas reuniões semanais para acompanhamento das atividades, identificação de impedimentos, revisão das prioridades e definição dos próximos itens do backlog. O Kanban será utilizado de maneira adaptada, sem a necessidade de introduzir papéis ou cerimônias que não agreguem valor ao contexto de uma equipe composta por apenas dois integrantes.

O desenvolvimento partirá do marco de 20% já entregue e seguirá para V0 e V1; V2 e V3 permanecerão como possibilidades posteriores. A V0 será utilizada principalmente para reduzir os riscos técnicos relacionados à introspecção do PostgreSQL, preparação de contexto, geração de consultas SQL e execução segura. Após a validação desse fluxo, o projeto avançará para a V1, correspondente ao MVP e à principal entrega prevista no TCC, incluindo a integração do backend com uma interface gráfica.

Essa organização também se relaciona com os princípios de customização de processos, ou *tailoring*, estudados durante a disciplina. A implementação gradual de um processo ou solução permite que áreas prioritárias sejam tratadas primeiro, evitando a tentativa de implantação de todos os elementos simultaneamente. No contexto deste projeto, essa ideia será aplicada por meio da divisão da solução em incrementos progressivos, nos quais cada versão adicionará novas capacidades a uma base previamente validada.

Outro princípio considerado será a seleção de artefatos de acordo com o valor que fornecem ao desenvolvimento. Considerando o tamanho da equipe, serão priorizados os documentos e artefatos necessários para apoiar a comunicação, rastreabilidade, implementação e avaliação da solução, evitando a produção de documentação cujo custo de elaboração e manutenção não seja proporcional ao benefício obtido.

Entre os principais artefatos previstos encontram-se:

- backlog do produto;
- histórias de usuário;
- especificação de requisitos;
- documentação da arquitetura;
- diagramas considerados necessários para representar a solução;
- código-fonte versionado;
- casos e resultados de testes;
- relatório técnico do projeto.

A divisão de responsabilidades também será adaptada às características da equipe. Ambos os integrantes participarão do desenvolvimento, especialmente das atividades relacionadas ao backend e ao fluxo central da aplicação. Entretanto, um dos integrantes terá maior concentração nas atividades de organização, documentação e apresentação do projeto, enquanto o outro terá maior concentração na implementação.

Essa divisão não será rígida, e as principais decisões técnicas e atividades de desenvolvimento permanecerão compartilhadas entre os integrantes. Dessa forma, a distribuição de responsabilidades poderá ser representada por uma matriz simplificada, evitando a criação de uma estrutura organizacional desnecessariamente complexa.

O código-fonte será versionado utilizando Git e armazenado em um repositório no GitHub. O versionamento permitirá o acompanhamento das alterações realizadas, a integração do trabalho dos dois integrantes e a recuperação de versões anteriores quando necessário.

A validação do sistema ocorrerá progressivamente durante o desenvolvimento. Será criado um conjunto controlado de perguntas cujos resultados esperados sejam previamente conhecidos, permitindo avaliar o comportamento do sistema diante de diferentes níveis de complexidade.

A avaliação priorizará a correção do resultado retornado, e não necessariamente a igualdade textual entre o SQL produzido pelo sistema e uma consulta de referência.

Essa decisão considera que consultas SQL sintaticamente diferentes podem produzir resultados semanticamente equivalentes.

Entre as principais métricas previstas para a avaliação estão:

- taxa de respostas corretas;
- taxa de consultas SQL executadas com sucesso;
- taxa de bloqueio correto de operações proibidas;
- quantidade de consultas que exigiram uma ou mais tentativas de correção;
- número médio de tentativas realizadas;
- latência entre a solicitação do usuário e a apresentação do resultado;
- quantidade de tokens utilizados, quando essa informação for disponibilizada pelo provedor;
- custo estimado por consulta, quando aplicável.

A comparação entre diferentes modelos de linguagem poderá ser realizada internamente durante o desenvolvimento para apoiar a escolha da alternativa que apresente melhor equilíbrio entre qualidade, custo e latência. Entretanto, essa comparação não será considerada, inicialmente, um experimento formal do TCC.

2.4 Tecnologias

A seleção das tecnologias previstas para o desenvolvimento foi orientada pela compatibilidade com o problema proposto, maturidade das ferramentas, facilidade de integração, disponibilidade de bibliotecas e conhecimento prévio da equipe.

A solução será estruturada em duas camadas principais de execução: uma aplicação responsável pela interface com o usuário, construída com React, Vite, TypeScript e Electron, e um backend desenvolvido em Python e FastAPI, responsável pela lógica de negócio, introspecção do banco de dados, integração com modelos de linguagem, validação e execução das consultas.

2.4.1 Python

Python será utilizado como principal linguagem de programação do backend. A linguagem possui um amplo ecossistema de bibliotecas voltadas à integração com bancos de dados, construção de APIs, manipulação de dados e comunicação com

modelos de linguagem, características diretamente relacionadas às necessidades da aplicação.

Além disso, sua utilização permitirá concentrar no mesmo ambiente as etapas de introspecção do banco, preparação de contexto, integração com provedores de inteligência artificial e execução das consultas.

2.4.2 FastAPI

O FastAPI será utilizado para a construção da camada de API e para a orquestração dos diferentes módulos do sistema.

O framework será responsável por disponibilizar os serviços utilizados pela interface, receber as solicitações dos usuários e coordenar etapas como conexão com o banco, introspecção, processamento da pergunta, comunicação com o modelo de linguagem e execução das consultas.

2.4.3 Pydantic

O Pydantic será utilizado para definição e validação das estruturas de dados utilizadas pela aplicação.

Sua utilização permitirá estabelecer contratos explícitos para entradas e saídas da API, contribuindo para a validação dos dados recebidos e para a organização da comunicação entre os diferentes componentes do sistema.

2.4.4 PostgreSQL

O PostgreSQL será o Sistema Gerenciador de Banco de Dados oficialmente suportado no escopo inicial do projeto.

A escolha de um único SGBD para o MVP permitirá concentrar os esforços no desenvolvimento do módulo de descoberta, na compreensão de sua estrutura e na geração correta de consultas compatíveis com seu dialeto SQL.

A aplicação não ficará limitada a um banco previamente configurado. O usuário poderá informar os dados de conexão de diferentes bancos PostgreSQL, desde que as credenciais utilizadas possuam acesso às estruturas que se deseja consultar.

2.4.5 Acesso ao PostgreSQL e Validação SQL

O núcleo entregue nos 20% utiliza psycopg para acessar o PostgreSQL e consultar seus metadados de maneira determinística, considerando os objetos acessíveis

à role. O SQLAlchemy poderá ser empregado se necessário, mas não será uma dependência arquitetural obrigatória.

O SQLGlot será utilizado para analisar a estrutura sintática da SQL no dialeto PostgreSQL e aplicar uma política conservadora de leitura. Essa análise não comprova, isoladamente, segurança ou correção semântica: será complementada pelos privilégios do banco, pela confirmação humana e pelos limites de execução da V0 e da V1.

2.4.6 React, Vite e TypeScript

A interface da aplicação será desenvolvida utilizando React em conjunto com TypeScript, tendo o Vite como ferramenta de desenvolvimento e *build* do frontend.

O React será utilizado para estruturar a interface e os componentes responsáveis pela interação do usuário com o sistema. A escolha busca facilitar a construção de uma interface modular, permitindo separar elementos relacionados à configuração da conexão, envio de perguntas, apresentação de erros e visualização dos resultados.

O TypeScript será adotado como linguagem principal do frontend, adicionando tipagem estática ao desenvolvimento e contribuindo para a definição de contratos mais claros entre os componentes da interface e os dados recebidos do backend.

O Vite será utilizado como ferramenta de desenvolvimento, empacotamento e geração dos arquivos utilizados pela interface. Sua utilização permitirá manter um fluxo de desenvolvimento simplificado e integrado ao ecossistema React e TypeScript.

Na V1, a interface terá como objetivo disponibilizar um fluxo simples para que o usuário possa configurar a conexão com o banco PostgreSQL, autorizar ou recusar a utilização de amostras de dados, formular perguntas em linguagem natural e visualizar os resultados retornados.

Funcionalidades mais avançadas relacionadas à interpretação textual e visualização gráfica dos resultados poderão ser adicionadas respectivamente nas versões V2 e V3.

2.4.7 Electron

O Electron será utilizado para empacotar a interface desenvolvida com React, Vite e TypeScript como uma aplicação desktop executável no computador do usuário.

A adoção dessa tecnologia está relacionada principalmente à necessidade de permitir que a aplicação seja executada próxima ao ambiente onde o banco de dados está disponível. Em diferentes cenários, o PostgreSQL que será consultado poderá estar instalado localmente no computador do usuário ou disponível apenas por meio

de uma rede interna, o que tornaria menos adequada a dependência de uma aplicação exclusivamente hospedada em ambiente externo.

Por meio do Electron, será possível reutilizar tecnologias tradicionalmente empregadas no desenvolvimento web para construir a interface gráfica da aplicação, mantendo ao mesmo tempo uma experiência de software desktop.

A aplicação desktop atuará como camada de interação com o usuário, enquanto o processamento relacionado à introspecção do banco, integração com modelos de linguagem, geração de SQL, validação e execução das consultas permanecerá concentrado no backend desenvolvido em Python e FastAPI.

Essa separação permitirá manter responsabilidades distintas entre interface e processamento, ao mesmo tempo em que possibilitará a distribuição da solução como uma aplicação executável localmente.

2.4.8 Modelos de Linguagem

Os modelos de linguagem serão responsáveis pelas etapas que exigem interpretação semântica, principalmente a compreensão da solicitação do usuário e a geração das consultas SQL.

A integração utilizará o contrato `LLMClientPort`, com adaptadores específicos para cada serviço e sem dependência do núcleo em relação ao SDK de um fornecedor.

O Ollama local já integra o núcleo dos 20%. A V1 também oferecerá ao menos uma API externa com chave do usuário; o fornecedor remoto e os modelos definitivos serão escolhidos por testes de qualidade, latência e custo. O adaptador OpenAI representado nos diagramas é uma possibilidade de implementação, sem fechar a escolha do fornecedor.

A solução será projetada de maneira a evitar acoplamento excessivo a um único modelo ou provedor, permitindo que essa escolha seja alterada sem exigir modificações significativas nas demais camadas da aplicação.

A descoberta da estrutura do banco permanecerá separada desse processo. Informações que podem ser obtidas deterministicamente, como nomes de tabelas, colunas, tipos e relacionamentos, serão extraídas pelo módulo de introspecção e fornecidas ao modelo como contexto, em vez de delegar ao modelo a responsabilidade de descobrir essas informações.

2.4.9 Git e GitHub

Git será utilizado como sistema de controle de versão do código-fonte, enquanto o GitHub será utilizado para hospedagem do repositório e colaboração entre os

integrantes.

Essas ferramentas permitirão registrar o histórico de alterações, integrar o trabalho realizado pela equipe e manter rastreabilidade sobre a evolução da implementação.

2.4.10 Execução Local

A execução local será adotada como estratégia principal para o MVP.

Essa decisão está relacionada à característica do sistema de interagir com bancos PostgreSQL que poderão estar instalados no próprio computador do usuário ou disponíveis apenas em redes internas. Dessa forma, a execução da aplicação no ambiente do usuário reduz a necessidade de expor o banco de dados a serviços externos apenas para possibilitar a conexão.

O instalador Electron incluirá o backend Python empacotado e o iniciará automaticamente, dispensando uma instalação separada de Python pelo usuário. A API local ficará restrita a 127.0.0.1, protegida por token efêmero de sessão; a interface utilizará uma ponte controlada para acesso aos segredos, sem expô-los ao renderer.

O núcleo de consulta não será hospedado publicamente. O serviço em nuvem da V1 se limitará às contas, instalações e métricas agregadas do produto, sem receber o conteúdo dos bancos ou das conversas. A política de sessão offline ainda será detalhada.

3 RESULTADOS E DISCUSSÃO

Até 10 de novembro de 2026, o resultado implementado corresponde ao marco de 20%, entregue na semana anterior. O núcleo integra descoberta autorizada do PostgreSQL, geração de SQL via Ollama e validação independente com SQLGlot, acessíveis por CLI e API FastAPI. A proposta e a decisão do validador são apresentadas ao usuário, sem execução da SQL gerada.

O registro de evidências da implementação, datado de 3 de novembro de 2026, informa a aprovação de 92 testes, incluindo integração com PostgreSQL real, e uma demonstração com o modelo local `qwen3.5:9b`. Esses resultados dizem respeito ao ambiente e aos casos registrados naquela verificação; não constituem uma nova execução de testes nesta revisão documental, nem garantia geral de correção ou segurança de produção.

Contexto por conexão, correção controlada, confirmação e execução de consultas permanecem como entregas previstas para a V0. A interface desktop, a entrevista completa, a persistência local e o Control Plane pertencem à V1. Sua avaliação será documentada após a implementação, conforme o planejamento apresentado neste relatório.

4 CONCLUSÕES PARCIAIS

A etapa de pré-projeto permitiu delimitar a proposta de um sistema de apoio à análise de dados por meio de perguntas em linguagem natural, com PostgreSQL como sistema gerenciador de banco de dados inicialmente suportado. Foram estabelecidos o fluxo principal da aplicação, os requisitos e a organização do desenvolvimento em incrementos. A V0 será utilizada como prova de conceito para verificar a viabilidade técnica da solução, enquanto a V1 constituirá o Produto Mínimo Viável e o compromisso de entrega do trabalho.

O marco de 20% já foi entregue e demonstrou a integração entre introspecção do PostgreSQL, geração via Ollama e validação independente, sem executar a SQL proposta. A V0, prevista para a semana de 9 a 15 de novembro de 2026, ampliará esse núcleo com contexto por conexão, confirmação e execução controlada. A V1 permanece como entrega do semestre seguinte. A correção geral das respostas, a segurança da execução e a experiência desktop ainda dependerão das avaliações dos próximos incrementos.

4.1 Papel do Contexto na Solução

Uma das principais conclusões do planejamento é que o valor esperado da solução depende fortemente da qualidade do contexto fornecido ao modelo de linguagem. O modelo precisa relacionar os termos utilizados na pergunta com as estruturas e os dados disponíveis no banco, de modo que a geração de SQL corresponda à intenção do usuário.

Por esse motivo, o módulo de descoberta e introspecção do banco, associado à preparação do contexto, constitui um dos componentes centrais da proposta. Sua responsabilidade será identificar esquemas, tabelas, colunas, tipos, chaves e relacionamentos, organizando essas informações para utilização nas etapas de interpretação e geração de consultas. A extração dessas informações estruturais será determinística, sem delegar ao modelo a descoberta da estrutura do banco.

Entretanto, a estrutura do banco não descreve necessariamente todo o significado de negócio dos dados. Nomes, comentários e amostras poderão auxiliar a interpretação, mas códigos internos e conceitos como faturamento, pedido concluído ou cliente ativo podem depender de regras não explicitadas nas tabelas. A suficiência do contexto disponível será, portanto, uma questão a investigar, sem pressupor que a introspecção permita reconstruir automaticamente todas as regras de negócio.

Para a V0 e a V1, amostras serão opcionais e dependerão de autorização por tabela, limitadas a cem registros com todas as colunas acessíveis à role; o envio remoto exigirá consentimento separado. A recusa não impedirá o uso de metadados e do contexto de negócio validado. Na V0, cada conexão poderá receber um `context.md` manual; na V1, uma entrevista de até quinze perguntas apoiará sua construção, sempre mediante confirmação do usuário. Resultados de consultas não serão reutilizados como contexto da LLM.

4.2 Definições Consolidadas e Pendências

A implementação utilizará Python e FastAPI no backend e React, Vite e TypeScript na interface. O Electron será empregado para disponibilizar a interface como aplicação desktop, integrada ao backend executado localmente. O backend será incluído no instalador. O núcleo de consulta permanecerá local; cadastro, login e registro básico de contas e instalações serão atendidos por um Control Plane separado em nuvem, sem conteúdo das consultas.

A segurança será apoiada no uso de credenciais com permissões somente de leitura, na validação independente da consulta gerada e no controle da execução pelo backend. A avaliação realizada pelo modelo sobre a solicitação terá caráter complementar e não substituirá os mecanismos de restrição do banco e de validação da aplicação.

O processamento de uma pergunta admitirá, no máximo, três tentativas no total: uma tentativa inicial e até duas correções automáticas. Cada consulta produzida deverá passar novamente pela validação e pela confirmação explícita do usuário antes da execução, vinculada à SQL exata e à conexão, com transação somente leitura, timeout de trinta segundos e limite de retorno. Esgotado esse limite, a aplicação encerrará a operação de forma controlada e informará a falha ao usuário.

Ollama local e API externa com chave do usuário são as duas categorias previstas para a V1; o fornecedor remoto, os modelos definitivos, a política de sessão offline e as metas finais de desempenho ainda serão definidos. As configurações e o histórico de conversas persistirão em SQLite, enquanto o contexto de negócio permanecerá em Markdown e os segredos salvos opcionalmente serão protegidos pelo mecanismo do Windows.

4.3 Maturidade do Escopo e Lições Aprendidas

A evolução do planejamento permitiu separar a demonstração do núcleo técnico, a prova de conceito executável e o produto desktop, evitando atribuir aos 20% funcionalidades que pertencem à V0 ou à V1. A integração inicial também consolidou

contratos reutilizáveis entre geração, banco e modelo de linguagem, sem exigir que o diagrama conceitual reproduza cada classe da implementação.

Outra definição central foi distinguir estrutura técnica de conhecimento de negócio: a introspecção descobre os objetos autorizados, mas as regras dependem de validação humana e pertencem à conexão. Essa separação orientou a escolha de um arquivo de contexto simples, de persistência local enxuta e de confirmação explícita antes de cada execução. O refinamento reforçou o princípio de que a IA propõe, o sistema aplica os controles e o usuário valida.

4.4 Avaliação Prevista e Evoluções

A avaliação utilizará um banco PostgreSQL fictício do domínio comercial, contendo clientes, produtos, pedidos e itens de pedido, com dados e regras de negócio previamente conhecidos pelos autores. As perguntas serão formuladas em português e seus resultados serão comparados com consultas de referência e respostas esperadas.

Será investigado o efeito do contexto de negócio validado por meio da comparação entre o esquema isolado e o mesmo esquema acrescido das regras registradas na entrevista. Modelo, instruções técnicas, perguntas, dados e política de correção serão mantidos constantes. A comparação não pressupõe melhoria obrigatória nem generalização para outros bancos. O desenho e as metas detalhadas serão revisados com o orientador; amostras, se usadas em estudo complementar, terão consentimento e controle próprios.

As versões V2 e V3 permanecerão como possibilidades de evolução, contemplando, respectivamente, explicações textuais dos resultados e visualizações gráficas. Catálogos semânticos, glossários de negócio e mecanismos de recuperação de contexto poderão ser estudados posteriormente, caso as limitações observadas indiquem sua necessidade. Essas funcionalidades não serão consideradas requisitos obrigatórios para a conclusão da V1.

5 CRONOGRAMA E GESTÃO DE PESSOAS

O planejamento registra o marco de 20% entregue na semana de 2 a 8 de novembro de 2026, a V0 prevista para a semana de 9 a 15 de novembro e o desenvolvimento, avaliação e entrega da V1 até junho de 2027, no semestre seguinte. A V0 verificará o fluxo de geração, confirmação e execução pela CLI, enquanto a V1 constituirá o MVP desktop e o compromisso de entrega do trabalho.

Os períodos apresentados são estimativas de planejamento e poderão ser ajustados conforme a disponibilidade da equipe, os resultados dos incrementos e o calendário acadêmico. A data específica da banca técnica da disciplina GPD2 será incorporada quando estiver disponível.

5.1 Gestão de Pessoas

A equipe é composta por Gustavo da Silva Coelho e Raissa Carla Ferreira da Silva. Gustavo concentrará sua atuação no desenvolvimento e na integração técnica, com disponibilidade de até seis horas semanais. Suas responsabilidades principais serão a implementação do backend em Python e FastAPI, a integração com PostgreSQL e com o modelo de linguagem, a construção da interface em React e o empacotamento com Electron, além dos testes técnicos e da correção de falhas.

Raissa concentrará sua atuação na organização do cronograma, acompanhamento das atividades, consolidação da documentação e preparação da apresentação, com disponibilidade estimada entre seis e oito horas semanais. Também apoiará a organização das evidências de avaliação e a comunicação da proposta, de suas contribuições e dos resultados obtidos.

A divisão representa a concentração principal das atividades, sem impedir a colaboração entre os integrantes. Ambos participarão das decisões de escopo e arquitetura, da definição das perguntas de avaliação, da conferência dos resultados esperados, da revisão do relatório e da preparação da defesa.

O projeto contará com a orientação do professor Giovani Fonseca Ravagnani Disperati. Os encontros com o orientador serão acordados conforme sua disponibilidade e os marcos do projeto. A avaliação acadêmica envolverá o orientador e os professores responsáveis pelas etapas do trabalho.

O acompanhamento será realizado por meio de um processo baseado em Kanban, com backlog priorizado e reuniões semanais entre os integrantes. As tarefas deverão indicar responsável, resultado esperado e situação, utilizando estados como a

fazer, em andamento, em revisão e concluída. O código-fonte será versionado com Git e mantido no GitHub. A documentação e os testes acompanharão o desenvolvimento.

5.2 Cronograma das Atividades

A Figura 1 apresenta as atividades previstas até a Avaliação Final do Projeto Integrador. As linhas numeradas correspondem às descrições apresentadas a seguir. Cada marcação **X** indica o mês previsto de realização, sem representar uma atividade já concluída.

Figura 1 – Cronograma previsto de outubro de 2026 a junho de 2027

	Nº	Meses								
		2026			2027					
		OUT	NOV	DEZ	JAN	FEV	MAR	ABR	MAI	JUN
Atividades	1	X	X							
	2	X	X							
	3			X						
	4				X	X				
	5						X	X		
	6							X		
	7							X	X	
	8								X	
	9	X	X	X	X	X	X	X	X	X
	10									X

X = mês previsto para realização da atividade

Fonte: Elaborado pelos autores (2026).

1. **Consolidação do pré-projeto (outubro e novembro de 2026):** uniformizar escopo, requisitos, tecnologias, modelos e conclusões parciais, além de registrar o planejamento da equipe e da avaliação.
2. **Desenvolvimento da V0 (outubro e novembro de 2026):** implementar uma prova de conceito de conexão com PostgreSQL, introspecção, preparação de contexto, geração, validação e execução de consultas. Reutilizar o núcleo dos 20%, incluir um arquivo de contexto por conexão, confirmação explícita e três tentativas totais, utilizando um subconjunto de perguntas de teste. Amostras não serão obrigatórias para concluir a V0; qualquer uso respeitará os consentimentos e limites definidos.
3. **Revisão da V0 e planejamento da V1 (dezembro de 2026):** analisar os resultados iniciais, identificar pendências técnicas, refinar a escolha dos modelos e do fornecedor da API externa e organizar o backlog e o fluxo da interface.

4. **Desenvolvimento do núcleo da V1 (janeiro e fevereiro de 2027):** evoluir o núcleo da V0 com perfis de IA local e remota, entrevista por conexão, consentimentos e contexto validado, preservando geração, confirmação, execução restrita e tratamento de falhas. O fluxo admitirá três tentativas totais, incluindo a inicial.

5. **Interface e integração desktop (março e abril de 2027):** desenvolver e integrar a interface com React, Vite e TypeScript, disponibilizando conexão, perguntas e resultados tabulares, empacotar o backend FastAPI com Electron, incluir SQLite para histórico e configurações, proteger segredos e integrar o Control Plane básico de contas e instalações.

6. **Preparação da avaliação (abril de 2027):** consolidar o banco comercial fictício, as perguntas, as consultas de referência, os resultados esperados e os casos de segurança e falha. Congelar os dados e as regras de contexto validadas utilizados na comparação final.

7. **Estabilização e correções da V1 (abril e maio de 2027):** verificar os requisitos obrigatórios, executar testes de integração e corrigir os problemas identificados no fluxo principal e na avaliação.

8. **Avaliação comparativa e análise dos resultados (maio de 2027):** executar as perguntas com esquema e com esquema acrescido de contexto de negócio validado, registrar métricas e analisar respostas incorretas, tentativas de correção, custos e limitações.

9. **Documentação e acompanhamento (outubro de 2026 a junho de 2027):** atualizar o relatório, as decisões, o backlog, os registros de teste e as evidências do trabalho, com participação dos dois integrantes e maior concentração de Raissa na organização documental.

10. **Apresentação e entrega final (junho de 2027):** concluir a discussão dos resultados, revisar o relatório e a V1, preparar os materiais de apresentação e realizar a entrega para avaliação final.

A integração e a conclusão dos requisitos essenciais serão priorizadas antes da etapa final. Maio será reservado principalmente para avaliação e correções, enquanto junho será destinado à consolidação da entrega e à apresentação. As versões V2 e V3 permanecerão como possibilidades de evolução, sem integrar o compromisso obrigatório do cronograma.

5.3 Planejamento da Avaliação

A avaliação técnica será conduzida pelos autores em um banco PostgreSQL fictício do domínio comercial, contendo clientes, produtos, pedidos e itens de pedido. Será preparado um conjunto de trinta perguntas válidas em português, distribuídas entre dez consultas simples, oito agregações, oito consultas com junções e quatro consultas que envolvam significado de negócio. Adicionalmente, serão propostas dez verificações de solicitações proibidas e cinco verificações de falhas operacionais.

Cada pergunta válida terá sua interpretação esperada, uma consulta SQL de referência e o resultado esperado documentados. As perguntas utilizadas para ajustes durante o desenvolvimento serão separadas das utilizadas na avaliação final. O banco incluirá situações que permitam identificar erros, como pedidos cancelados, valores nulos e registros próximos aos limites dos períodos consultados.

Uma resposta será considerada correta quando os dados retornados corresponderem ao resultado esperado, incluindo filtros, valores, agrupamentos e ordenação quando solicitada. A igualdade textual com o SQL de referência não será exigida. Consultas que executem sem erro, mas respondam a uma intenção diferente, serão classificadas como incorretas.

Como desenho preliminar a revisar com o orientador, o efeito do contexto de negócio será investigado pela comparação entre duas configurações:

1. contexto contendo apenas os metadados estruturais do banco;
2. contexto contendo os mesmos metadados e as regras de negócio confirmadas na entrevista e registradas no arquivo da conexão.

O modelo, as instruções, as perguntas, os dados e a política de correção serão mantidos constantes. As regras validadas serão fixadas antes da comparação, e cada execução começará sem histórico de perguntas anteriores. Amostras não serão usadas nessa comparação principal; seu efeito poderá ser estudado separadamente, com consentimento e controle das demais condições. Prevê-se realizar três execuções por pergunta em cada configuração, totalizando cento e oitenta execuções de solicitações válidas, além das verificações de segurança e falha. Cada execução respeitará o limite de três tentativas totais.

Serão registradas a taxa de respostas corretas na primeira tentativa e após as correções, a taxa de execução SQL bem-sucedida, a quantidade de tentativas, a latência e, quando disponíveis, o consumo de tokens e o custo estimado. Os resultados serão apresentados por configuração e grupo de perguntas. Os casos de segurança e falha serão contabilizados separadamente.

Como metas preliminares para discussão com o orientador, propõem-se:

- pelo menos 80% de respostas corretas às perguntas válidas na configuração com contexto de negócio, considerando até três tentativas totais;
- pelo menos 90% de solicitações válidas com execução SQL bem-sucedida nessa configuração, independentemente da correção semântica;
- nenhuma alteração do banco nas dez verificações de solicitações proibidas;
- encerramento controlado nas cinco verificações de falhas operacionais;
- ausência de envio de registros ao modelo quando o consentimento for recusado;
- realização do fluxo principal pela interface gráfica, sem utilização de terminal pelo usuário.

Essas metas deverão ser revisadas com o orientador antes da avaliação final e não constituem critérios oficiais da banca. A configuração com esquema isolado será avaliada separadamente, sem pressupor que a inclusão de contexto de negócio necessariamente melhore o desempenho. A meta de latência e os limites de custo serão definidos após as medições da V0.

A avaliação pelos professores considerará a coerência da proposta, a demonstração do sistema, a documentação e as evidências apresentadas. Essa participação não será tratada como um estudo de usabilidade com usuários finais.

REFERÊNCIAS

DENG, N.; CHEN, Y.; ZHANG, Y. Recent advances in text-to-SQL: A survey of what we have and what we expect. In: **Proceedings of the 29th International Conference on Computational Linguistics**. Gyeongju, Republic of Korea: International Committee on Computational Linguistics, 2022. p. 2166–2187. Disponível em: <https://aclanthology.org/2022.coling-1.190/>. 29, 30

PRESSMAN, S. R. **Engenharia de Software, uma Abordagem Profissional**. [S.l.: s.n.], 2016. 36

SOMMERVILLE, I. **Engenharia de Software**. [S.l.: s.n.], 2003. 36

SOMMERVILLE, I. **Engenharia de Software, 8 edição**. [S.l.: s.n.], 2007. 36

YU, T. et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: **Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing**. Brussels, Belgium: Association for Computational Linguistics, 2018. p. 3911–3921. Disponível em: <https://aclanthology.org/D18-1425/>. 29, 30

Apêndices

APÊNDICE A – MODELO DE REQUISITOS DO SISTEMA

Este apêndice apresenta a descrição textual dos requisitos do sistema e o modelo de casos de uso da versão V1 (MVP). Estão organizados os atores envolvidos, as regras de negócio que orientam o comportamento da aplicação, os casos de uso com suas descrições e a rastreabilidade entre requisitos, casos de uso e incrementos. Os requisitos funcionais (RF) e não funcionais (RNF) referenciados estão definidos na Seção 2.2.

A.1 Atores

O Quadro 1 apresenta os atores que interagem com o sistema.

Quadro 1 – Atores do sistema

Ator	**Tipo**	**Descrição**
Usuário	Primário	Pessoa que conhece o contexto dos dados, mas não necessariamente domina SQL. Configura a conexão, decide sobre o uso de amostras, formula perguntas e visualiza os resultados.
Provedor de IA	Secundário (externo)	Modelo local via Ollama ou serviço de API externa com chave do usuário que interpreta a solicitação e gera ou corrige a consulta SQL a partir do contexto recebido.
Banco PostgreSQL	Secundário (externo)	Banco de dados do usuário, acessado com as credenciais informadas, do qual são extraídos os metadados estruturais, as amostras autorizadas e os resultados das consultas.

Fonte: Elaborado pelos autores (2026).

A.2 Regras de Negócio

As regras de negócio a seguir complementam os requisitos e delimitam o comportamento esperado da aplicação.

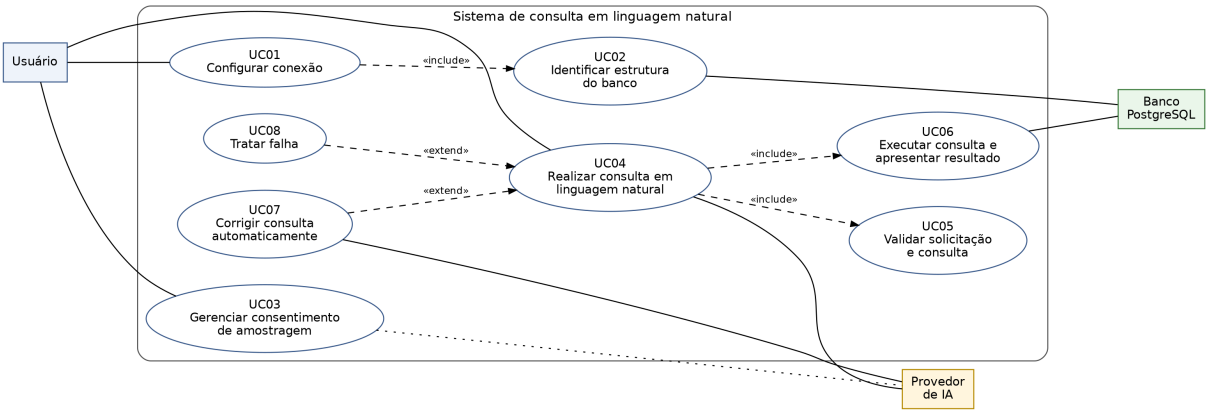
- **RN01 – Somente leitura:** apenas consultas de leitura, compostas por uma única instrução `SELECT`, podem ser executadas.
- **RN02 – Validação independente:** a resposta do modelo de linguagem não é tratada como confiável; toda consulta gerada passa por uma camada de validação sintática e de segurança independente do modelo antes da execução.

- **RN03 – Comandos bloqueados:** comandos como `INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER` e `TRUNCATE`, bem como quaisquer outros capazes de alterar registros ou estruturas, devem ser bloqueados antes da execução.
- **RN04 – Execução controlada:** a SQL validada será apresentada ao usuário e somente executada após confirmação explícita vinculada à proposta exata e à conexão, com revalidação, role restrita, transação somente leitura, timeout de trinta segundos e limite de retorno.
- **RN05 – Estrutura determinística:** os metadados estruturais (esquemas, tabelas, colunas, tipos, chaves primárias e estrangeiras) são obtidos pelo módulo de introspecção, e não por inferência do modelo de linguagem.
- **RN06 – Consentimento explícito:** amostras exigirão autorização por tabela e consentimento separado para envio remoto. Resultados de consultas não serão enviados à LLM.
- **RN07 – Limite de amostragem:** quando autorizada, cada amostragem conterá no máximo cem registros por tabela, incluindo todas as colunas acessíveis à role.
- **RN08 – Limite de correções:** cada pergunta admitirá uma geração inicial e até duas correções, totalizando três tentativas; cada nova SQL será validada e, para execução, confirmada novamente pelo usuário.
- **RN09 – Alcance definido pelas credenciais:** o sistema considera apenas as estruturas acessíveis às credenciais fornecidas pelo usuário.
- **RN10 – Rastreabilidade técnica:** latência, erros, número de tentativas e execução das consultas são registrados para apoiar a avaliação do sistema.
- **RN11 – Contexto isolado:** cada conversa terá uma conexão ativa. Seu arquivo de contexto reunirá somente regras confirmadas, sem compartilhamento automático com outras conexões.
- **RN12 – Persistência mínima:** SQLite armazenará configurações, consentimentos e histórico de perguntas/SQLs; segredos salvos opcionalmente ficarão protegidos pelo mecanismo do Windows e resultados tabulares não serão persistidos por padrão.

A.3 Casos de Uso

A Figura 2 representa o fluxo central de consulta do MVP, sem pretensão de enumerar todas as funções da V1. O Quadro 2 e as descrições seguintes complementam esse recorte com contexto, perfis de IA, persistência e conta do produto.

Figura 2 – Diagrama de casos de uso – fluxo central do MVP (V1)



Fonte: Elaborado pelos autores (2026).

Quadro 2 – Casos de uso e requisitos atendidos

Código	Caso de uso	Requisitos	Ator(es)
UC01	Configurar conexão com o banco	RF01	Usuário
UC02	Identificar estrutura do banco	RF02, RF03	Banco PostgreSQL
UC03	Gerenciar consentimento de amostragem	RF04, RF05	Usuário, Provedor de IA
UC04	Realizar consulta em linguagem natural	RF06, RF08	Usuário, Provedor de IA
UC05	Validar solicitação e consulta	RF07, RNF01	(interno)
UC06	Executar consulta e apresentar resultado	RF09, RF10	Banco PostgreSQL
UC07	Corrigir consulta automaticamente	RF11	Provedor de IA
UC08	Tratar falha	RF12, RNF06	Usuário
UC09	Preparar contexto de negócio	RF13	Usuário, Provedor de IA
UC10	Configurar perfil de IA	RF14	Usuário, Provedor de IA
UC11	Consultar histórico e preferências	RF15	Usuário
UC12	Acessar conta do produto	RF16	Usuário, Control Plane

Fonte: Elaborado pelos autores (2026).

A.3.1 UC01 – Configurar conexão com o banco

Objetivo: permitir que o usuário informe os dados de acesso a um banco PostgreSQL.

Ator: Usuário.

Pré-condição: a aplicação está em execução.

Pós-condição: conexão estabelecida e estrutura do banco identificada (UC02).

Fluxo principal:

1. O usuário informa os dados de conexão (servidor, porta, banco e credenciais de uma role preexistente de menor privilégio).
2. O sistema tenta estabelecer a conexão.
3. O sistema aciona a identificação da estrutura do banco (UC02).
4. O sistema disponibiliza a estrutura da conexão e permite preparar seu contexto de negócio (UC09) e selecionar um perfil de IA (UC10). Na V1, a configuração não sensível será persistida localmente.

Fluxo alternativo: se a conexão falhar (dados incorretos ou banco inacessível), o sistema informa o erro ao usuário e permite corrigir os dados (RNF06).

A.3.2 UC02 – Identificar estrutura do banco

Objetivo: levantar automaticamente a estrutura acessível do banco, sem depender do modelo de linguagem.

Ator: Banco PostgreSQL.

Pré-condição: conexão estabelecida (UC01).

Pós-condição: representação estruturada com esquemas, tabelas, colunas, tipos, chaves primárias e estrangeiras disponível para composição do contexto.

Fluxo principal:

1. O sistema lista os esquemas acessíveis às credenciais informadas (RF02).
2. Para cada esquema, o sistema obtém tabelas, colunas, tipos de dados, chaves primárias e chaves estrangeiras (RF03).
3. O sistema organiza as informações em uma representação estruturada.

A.3.3 UC03 – Gerenciar consentimento de amostragem

Objetivo: permitir que o usuário autorize ou recuse o envio de amostras de registros ao provedor de IA.

Ator: Usuário.

Pré-condição: estrutura do banco identificada (UC02).

Pós-condição: decisão registrada por tabela, com consentimento separado para envio remoto; na V1, os consentimentos essenciais serão persistidos localmente.

Fluxo principal:

1. O sistema informa que as amostras podem conter dados sensíveis e solicita autorização explícita (RF04).
2. O usuário autoriza a utilização de amostras da tabela e decide separadamente sobre envio a um provedor remoto.
3. Quando necessário, o sistema coleta amostras de, no máximo, cem registros por tabela, incluindo todas as colunas acessíveis à role (RF05) e as inclui no contexto.

Fluxo alternativo: se o usuário recusar, nenhum registro é lido para composição do contexto e o sistema opera com os metadados e o contexto de negócio permitido, sem amostras (RNF03).

A.3.4 UC04 – Realizar consulta em linguagem natural

Objetivo: obter dados do banco a partir de uma pergunta escrita em linguagem natural.

Atores: Usuário e Provedor de IA.

Pré-condição: banco conectado e estrutura identificada.

Pós-condição: resultado apresentado em formato tabular, ou falha informada de forma controlada.

Fluxo principal:

1. O usuário digita uma pergunta em linguagem natural (RF06).
2. O sistema prepara o contexto com o esquema autorizado, o arquivo de negócio da conexão, a pergunta e pares anteriores de pergunta/SQL da mesma sessão, respeitando os limites definidos; amostras só serão usadas quando autorizadas.
3. O sistema verifica se a solicitação é compatível com o escopo de leitura (UC05).
4. O sistema solicita ao provedor de IA a geração da consulta SQL (RF08).
5. O sistema valida a consulta gerada (UC05).
6. O sistema apresenta a SQL validada e solicita confirmação explícita para essa proposta e conexão.

7. Se confirmada, o sistema revalida, executa e apresenta o resultado em tabela (UC06).

Fluxos alternativos:

A1. *Solicitação incompatível:* se a pergunta exigir inserção, atualização, exclusão ou alteração estrutural, o sistema não prossegue para a execução e informa o motivo ao usuário (RF07).

A2. *Consulta reprovada na validação:* a consulta não é executada; o sistema pode solicitar nova geração dentro do limite de tentativas (UC07) ou, esgotado o limite, tratar a falha (UC08).

A3. *Erro de geração ou execução:* o sistema poderá acionar a correção dentro do limite total (UC07), sem executar automaticamente a nova SQL.

A4. *Execução recusada:* o usuário poderá copiar a SQL ou encerrar o fluxo; nenhuma consulta será executada.

A.3.5 UC05 – Validar solicitação e consulta

Caso de uso incluído em UC04. O sistema verifica a intenção da pergunta (RF07) e, após a geração, analisa a consulta produzida por uma camada de validação independente do modelo, aprovando apenas uma instrução `SELECT` e bloqueando comandos de escrita ou de alteração estrutural (RN01 a RN03; RNF01).

A.3.6 UC06 – Executar consulta e apresentar resultado

Caso de uso incluído em UC04. Após confirmação explícita da proposta exata e da conexão, o sistema revalida e executa a SQL com role restrita, transação somente leitura, timeout de trinta segundos e limite de linhas (RF09; RN04), processa o resultado e o apresenta ao usuário em formato tabular na interface gráfica (RF10), registrando latência e demais informações de execução (RNF07).

A.3.7 UC07 – Corrigir consulta automaticamente

Caso de uso que estende UC04. Quando uma consulta permitida falha na geração ou na execução, o sistema envia o erro ao provedor de IA e obtém uma nova versão da consulta, respeitando três tentativas totais, incluindo a geração inicial (RF11; RN08). Uma nova SQL exige validação e nova confirmação antes da execução. Cada tentativa é registrada para fins de avaliação (RN10).

A.3.8 UC08 – Tratar falha

Caso de uso que estende UC04. Esgotado o limite de tentativas, ou diante de falhas de conexão ou de comunicação com o provedor de IA, o sistema encerra a operação de maneira controlada e informa a falha ao usuário, sem executar consultas não aprovadas (RF12; RNF06).

A.3.9 UC09 – Preparar contexto de negócio

Na V0, o usuário poderá preparar manualmente o `context.md` da conexão. Na V1, a entrevista utilizará o esquema autorizado para formular até quinze perguntas, com alternativas sugeridas, resposta livre e possibilidade de correção. Somente interpretações confirmadas serão gravadas. O usuário poderá revisar o arquivo e refazer a entrevista; o `ContextBuilder` o lerá integralmente, respeitando os limites de contexto, sem RAG ou seleção semântica (RF13; RN11).

A.3.10 UC10 – Configurar perfil de IA

O usuário configurará e testará endpoint e modelo do Ollama local ou de uma API externa com chave própria. A V1 persistirá a configuração não sensível e, opcionalmente, uma referência ao segredo protegido pelo mecanismo do Windows. Os adaptadores utilizarão o mesmo contrato `LLMClientPort` (RF14; RNF05).

A.3.11 UC11 – Consultar histórico e preferências

A V1 permitirá recuperar conversas, perguntas, SQLs e estados de validação, além de manter nome, preferências e configurações do usuário. O histórico ficará associado à conexão, sem armazenar resultados tabulares por padrão. Consultar o histórico não autorizará executar novamente uma SQL nem enviará automaticamente conversas antigas à LLM (RF15; RN12).

A.3.12 UC12 – Acessar conta do produto

O usuário realizará cadastro e login pelo Control Plane, que registrará contas e instalações e receberá somente métricas básicas agregadas de uso. O serviço não receberá esquemas, perguntas, SQLs, arquivos de contexto ou resultados. A política de sessão e operação offline ainda será detalhada, preservando a intenção de não depender continuamente da nuvem para consultas locais (RF16).

A.4 Rastreabilidade dos Requisitos

O Quadro 3 relaciona os requisitos funcionais aos casos de uso e aos incrementos em que são implementados. O marco de 20% já contempla descoberta, geração e validação sem execução. A V0 acrescentará contexto, correção, confirmação e execução pela CLI; a V1 integrará a interface desktop e as funções de produto. A tabela indica os marcos previstos para o atendimento completo de cada requisito.

Quadro 3 – Rastreabilidade dos requisitos funcionais

Req.	**Descrição resumida**	**Caso de uso**	**Incremento**
RF01	Conexão com banco de dados	UC01	V0 e V1
RF02	Identificação de esquemas	UC02	V0 e V1
RF03	Introspecção estrutural	UC02	V0 e V1
RF04	Consentimento para amostragem	UC03	V1
RF05	Limitação das amostras	UC03	V1
RF06	Entrada em linguagem natural	UC04	V0 e V1
RF07	Validação da intenção	UC04, UC05	V0 e V1
RF08	Geração de consulta SQL	UC04	V0 e V1
RF09	Execução da consulta	UC06	V0 e V1
RF10	Resultado tabular na interface gráfica	UC06	V1
RF11	Correção controlada de consultas	UC07	V0 e V1
RF12	Tratamento de falha	UC08	V0 e V1
RF13	Contexto por conexão; entrevista na V1	UC09	V0 e V1
RF14	Perfis de IA local e remota	UC10	V1
RF15	Histórico e configurações locais	UC11	V1
RF16	Conta e instalação	UC12	V1

Fonte: Elaborado pelos autores (2026).

A.5 Critérios de Verificação dos Requisitos Não Funcionais

O Quadro 4 indica como cada requisito não funcional será verificado durante a validação do sistema.

Quadro 4 – Critérios de verificação dos requisitos não funcionais

Req.	**Requisito**	**Forma de verificação**
RNF01	Segurança	Conjunto de solicitações e consultas proibidas; medição da taxa de bloqueio correto.
RNF02	Privacidade	Inspeção do contexto enviado ao provedor, verificando a ausência de registros sem consentimento.
RNF03	Independência da amostragem	Execução do fluxo principal com a amostragem recusada.
RNF04	Modularidade	Revisão da arquitetura quanto à separação entre introspecção, contexto, geração, validação, execução e apresentação.
RNF05	Baixo acoplamento ao modelo	Substituição do provedor de IA sem alteração dos demais módulos.
RNF06	Tratamento de erros	Simulação de falhas de conexão, de provedor, de geração e de execução.
RNF07	Observabilidade	Verificação dos registros de latência, erros, tentativas e execuções.
RNF08	Usabilidade	Execução do fluxo principal pela interface desktop, sem uso de terminal ou código-fonte.

Fonte: Elaborado pelos autores (2026).

APÊNDICE B – MODELO ESTRUTURAL DO SISTEMA

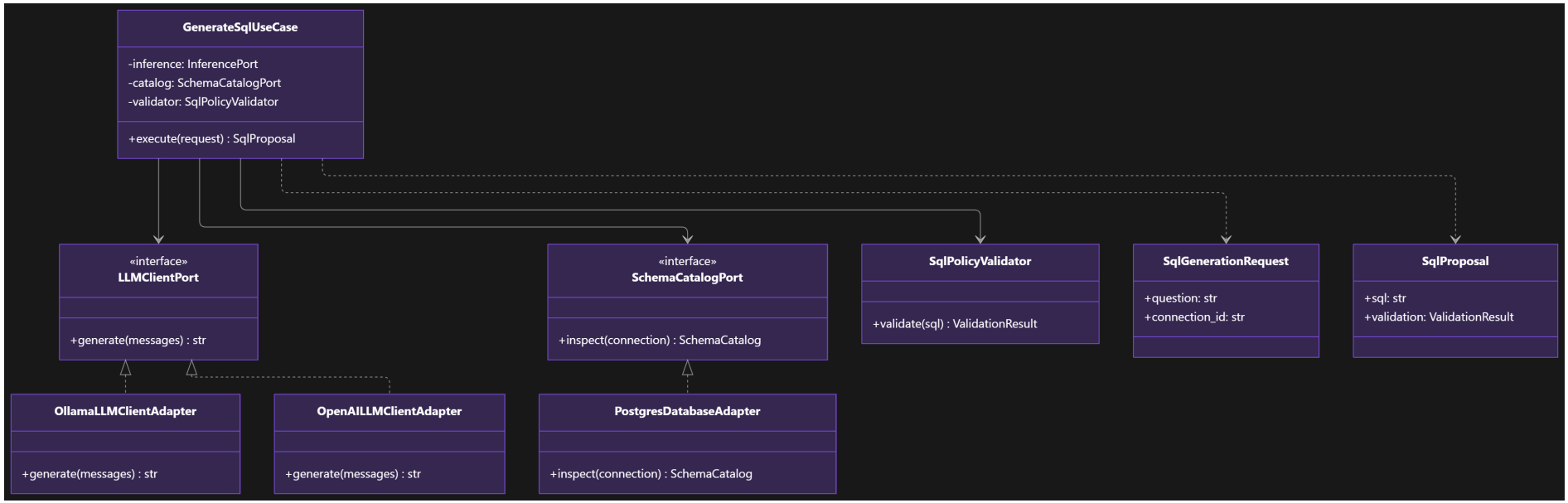
Este apêndice reúne duas visões complementares de classes para a V1: o núcleo de geração e validação de SQL e sua evolução com preparação de contexto e execução confirmada. Os diagramas são conceituais; orientam responsabilidades e dependências, sem exigir correspondência exaustiva com todas as classes, assinaturas ou módulos do código. Componentes previstos não devem ser interpretados como já implementados.

B.1 Núcleo de Geração e Validação SQL

A Figura 3 apresenta o núcleo coordenado pelo `GenerateSqlUseCase`. A descoberta do esquema utiliza o contrato `SchemaCatalogPort`, e a geração utiliza o `LLMClientPort`. A proposta passa pelo `SqlPolicyValidator`. Essa separação permite testar o núcleo e substituir integrações sem transferir ao modelo a responsabilidade de executar SQL. O PostgreSQL e os provedores de IA são acessados por adaptadores.

A imagem recebida conserva a referência antiga `InferencePort` em um atributo. Para o escopo e a nomenclatura atuais, esse atributo corresponde a `LLMClientPort`, já identificado como interface no próprio diagrama. O adaptador OpenAI ilustra a categoria de API externa do usuário; não determina o fornecedor remoto final e não significa que essa integração esteja concluída.

Figura 3 – Diagrama de classes – núcleo de geração e validação previsto para a V1



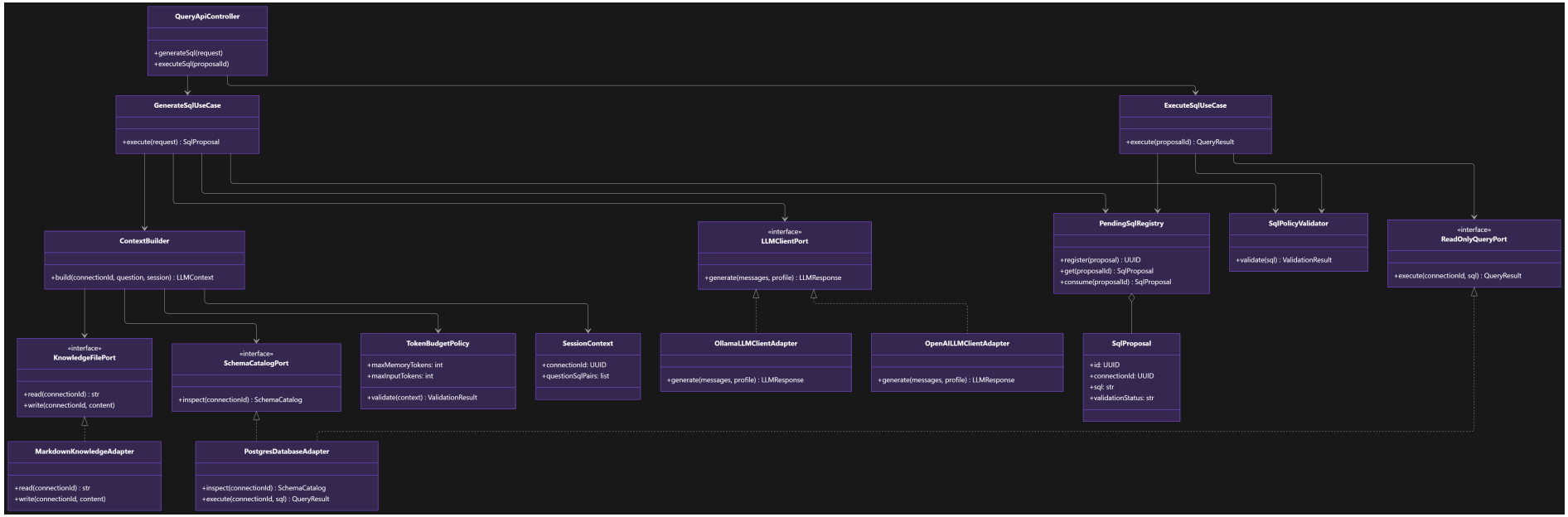
Fonte: Elaborado pelos autores (2026).

B.2 Contexto da Conexão e Execução Confirmada

A Figura 4 amplia o núcleo para a V1. O `ContextBuilder` reúne esquema autorizado, conhecimento validado do arquivo da conexão, pergunta e pares anteriores de pergunta/SQL da mesma sessão. O `KnowledgeFilePort` e seu adaptador Markdown representam o acesso ao `context.md`; o `TokenBudgetPolicy` verifica os limites de entrada. O diagrama não detalha todas as classes da entrevista: sua responsabilidade é obter confirmação humana antes de atualizar o arquivo.

Geração e execução permanecem separadas. O `PendingSqlRegistry` guarda temporariamente a proposta, e o `ExecuteSqlUseCase` somente utiliza uma aprovação vinculada à SQL exata e à conexão. Antes de chamar o `ReadOnlyQueryPort`, revalida a proposta; o adaptador PostgreSQL aplica a execução restrita. Uma proposta pendente não equivale a histórico persistente nem dispensa confirmação humana.

Figura 4 – Diagrama de classes – contexto por conexão e execução prevista para a V1



Fonte: Elaborado pelos autores (2026).

B.3 Considerações sobre o Modelo

O backend seguirá uma arquitetura hexagonal pragmática: contratos nas fronteiras com PostgreSQL, IA e arquivos, organizados em um único backend modular. Pydantic definirá contratos de dados e SQLGlot apoiará a validação estrutural; nenhuma dessas ferramentas substituirá os privilégios do PostgreSQL ou a confirmação do usuário. O núcleo dos 20% já utiliza psycopg; SQLAlchemy poderá ser adotado se necessário, sem constituir obrigação da arquitetura.

As classes de catálogo, contexto, proposta e resultado não serão automaticamente convertidas em tabelas SQLite. A persistência da V1 é um recorte próprio, apresentado no modelo de dados, e não uma réplica do banco consultado nem de todas as classes da aplicação.

APÊNDICE C – MODELO COMPORTAMENTAL DO SISTEMA

Texto de apresentação do Apêndice - Fluxograma ou Diagrama de Sequência ou Diagrama de Estados..

APÊNDICE D – MODELO DA ARQUITETURA DO SISTEMA

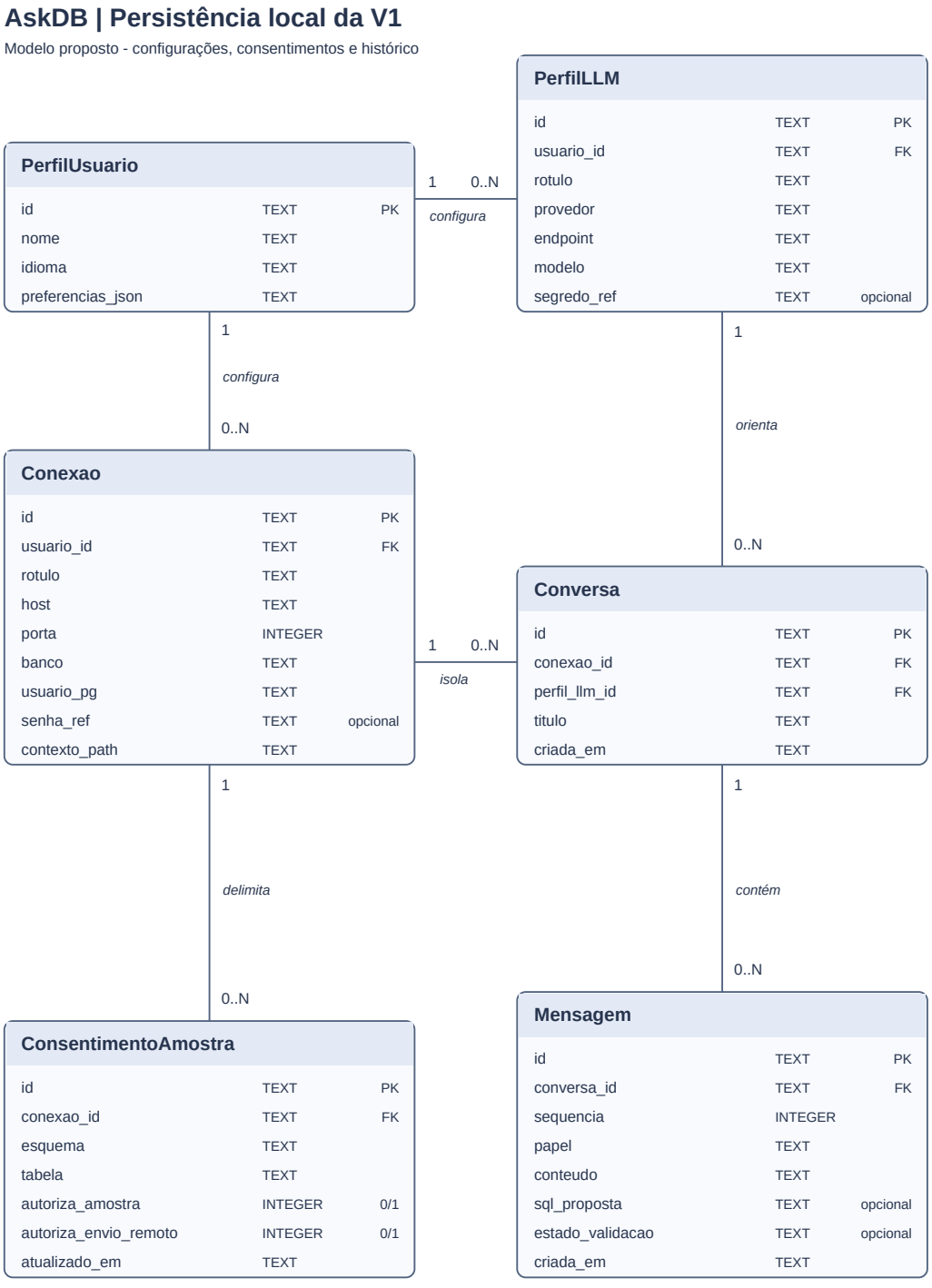
Texto de apresentação do Apêndice - Diagrama de blocos de alto nível ou Diagrama de Componentes.

APÊNDICE E – MODELO DE DADOS DO SISTEMA

O modelo entidade-relacionamento proposto para a V1 contempla apenas a persistência operacional do aplicativo no SQLite local. O histórico persistente, anteriormente em aberto, passa a integrar o escopo para permitir recuperar conversas e evitar reconfigurações. O modelo é um planejamento de implementação, não uma descrição de tabelas já existentes.

A Figura 5 apresenta seis entidades. `PerfilUsuario` mantém o nome e as preferências locais, sem reproduzir a autenticação do Control Plane. `Conexao` guarda o destino PostgreSQL, a identidade de acesso e o caminho do arquivo de contexto. `PerfilLLM` mantém provedor, endpoint e modelo. `Conversa` associa cada histórico a uma única conexão e ao perfil de IA selecionado; `Mensagem` registra a sequência de perguntas e propostas SQL, com estado de validação. `ConsentimentoAmostra` mantém somente decisões por tabela, sem armazenar os registros amostrados.

Figura 5 – DER proposto para a persistência local SQLite da V1



Fonte: Elaborado pelos autores (2026).

As relações indicam que um perfil local pode manter várias conexões e configurações de IA; cada conexão pode ter várias conversas e consentimentos. Cada conversa utiliza uma conexão e um perfil de IA, e pode conter várias mensagens ordenadas. A associação de conexão não muda durante a conversa. Mudanças de perfil de IA poderão iniciar outra conversa, preservando a identificação do perfil empregado no histórico. Essa organização não pressupõe gestão multiusuário ou permissões comerciais locais.

Os campos `senha_ref` e `segredo_ref` serão opcionais e conterão apenas identificadores do material protegido pelo Electron safeStorage/DPAPI, separado dos dados operacionais; senhas PostgreSQL e chaves de API não serão gravadas em texto puro no SQLite ou no Markdown. O nome do usuário local não se confunde com a role PostgreSQL nem com as credenciais da conta do produto.

Os consentimentos terão uma combinação única de conexão, esquema e tabela. Autorizar envio remoto dependerá também da autorização de amostragem; a recusa será o padrão na ausência de decisão. Apenas os identificadores necessários a essa decisão serão persistidos, sem modelar tabelas, colunas ou relações completas do catálogo consultado.

As mensagens persistirão perguntas, propostas SQL e estados de validação, sem linhas de resultados, amostras ou segredos. Recuperar uma conversa não significará reenviar automaticamente seu conteúdo à LLM, nem reaproveitar uma aprovação de execução anterior. A continuidade do contexto utilizará apenas pares de pergunta/SQL da sessão ativa, conforme a política de contexto; retenção e exclusão do histórico ainda serão detalhadas.

O conhecimento de negócio permanecerá em um `context.md` por conexão, e o catálogo será obtido por introspecção do PostgreSQL. Contas, instalações e métricas agregadas pertencem ao serviço em nuvem e não ampliam este DER local. O arquivo Mermaid que acompanha a figura permite revisar a modelagem durante a implementação.

APÊNDICE F – OUTRO APÊNDICE

Texto do Apêndice adicional.

Anexos

ANEXO A – TÍTULO DO ANEXO A

Texto do Anexo A.

ANEXO B – TÍTULO DO ANEXO B

Texto do Anexo B.

ANEXO C – TÍTULO DO ANEXO C

Texto do Anexo C.

ANEXO D – TÍTULO DO ANEXO D

Texto do Anexo D.

ANEXO E – TÍTULO DO ANEXO E

Texto do Anexo E.